Finite Address Component Replay and an SU(3) Singlet Audit

A conditional WRRA computation and a precise limit on address-color coupling

Wonsik Choi and Jeongin Choi
Independent Research, Seoul, Republic of Korea
20 October 2026 | Research note v0.6
Release: WRRA-COLOR-AUDIT-20261020-v0.6
Correspondence: janefather@gmail.com | ORCID 0009-0001-4263-9772

Abstract

We audit the particle/component branch of Minimal Computing Cosmology 3.0, using frozen source files and an unchanged finite-address calibration. A targeted rerun reproduces 427,892 odd-composite addresses, phenotype weight 0.050000000000000065, the reported conditional component fractions and an assembly-order total-variation distance of 0.20472100116369085. The declared antisymmetric three-color state is annihilated by all eight total color generators. Independent tests verify generator closure, collective SU(3) invariance, an exact one-dimensional common kernel and detection of symmetry-breaking controls. The declared replay, algebra, control and provenance checks pass; these overlapping tests are not independent observational evidence. Source inspection also establishes that the original color test accepts no arithmetic-state argument: three colors and their generators are explicitly supplied. We prove the consequent interface restriction: within three fundamental color factors and the exact total-color-singlet constraint, any joint address-color density operator factors as an address density operator times the fixed singlet projector. Thus this color sector cannot by itself encode address-dependent distinctions. The contribution is an executable WRRA interface audit and a conditional restriction on future coupling rules, rather than an arithmetic derivation of SU(3), confinement or the particle spectrum.

Assessment contract

Verification inputs: frozen finite-address parameters, adopted spectral inputs, inherited component decoder and charge table, and an explicitly chosen three-dimensional fundamental color representation.

WRRA transformation: phase-aware admission weights feed the cyclic prime-component assembly; the inherited charge and color contracts are checked; an independent kernel audit determines what address information the exact color sector can retain.

Outputs: component replay, charge neutrality, color-invariance residuals, exact singlet uniqueness and the factorization restriction.

Falsification conditions: failure of the fixed-input replay, normalization or integer conservation; nonzero total-color action on the proposed singlet; a second independent common-kernel vector; or an admissible joint singlet-sector state that does not factor as proved below. Physical application requires a specified additional mapping to observations.

1. Frozen inputs and original computation

WRRA abbreviates Worldline-Residue-Resource-Action. In this note it identifies a finite-state construction in which an address measure is filtered into sectors before a component interpretation is tested. The audited original Stage-3 claim is narrowly that the finite-source replay preserves assembly conservation, charge-neutral motifs and an inherited color singlet. The original README already labels the particle decoder conditional and spin and binding open. The development question is whether passing its existing color test is sufficient grounds to treat color as an arithmetic output or to delete address labels before a future decoder is implemented. We test that proposed inference; we do not claim that the original code made it or that an existing decoder discarded those labels.

The inspected repository is Wonsik-Choi-janefather/wrra-m-0.1 at commit 9d9daf21154e46e55b9fd0fd461c7c070156ab94. The Stage-2 handoff fixes N=1,015,000, alpha=1.8996877935161325 and h=1.4476744689338703. Address 1 is inactive. The address weights are proportional to n^(-alpha). Four adopted zeta-zero heights and eight admission frames determine the original odd-composite admission effect. These are disclosed construction inputs; no parameter is refitted in this audit.

```
p(n) = w(n) e_phi(n) / sum_m w(m) e_phi(m)
```

For odd composites, e_phi(n)=1-product_k[1-sigmoid(d_k(n)-h)], k=0,...,7, where d_k(n)=sum_j cos[gamma_j(log n+0.1k)]/2 and gamma=(14.134725141734695,21.022039638771556,25.01085758014569,30.424876125859512). Other addresses have zero phenotype effect. For each K-prime pool, begin with floor(n/sum(primes)) copies of every prime. Repeatedly traverse the selected order, taking one prime whenever the remaining residue permits it, until the residue is below 2. This preserves n=sum_j count_j prime_j+residue. The displayed candidate fraction is the p-weighted count of components labeled 5 or 47 divided by the p-weighted total component count, rather than a fraction of addresses.

Here sigmoid(x)=1/(1+exp(-x)). For a given order o, define a_j(o)=sum_n p(n) count_j(n,o), and the number-share distribution P_j(o)=a_j(o)/sum_l a_l(o), aligning components by ascending prime label. The reported K15 distance is TV=(1/2)sum_j |P_j(small)-P_j(large)|. It compares these component-number distributions, not address probabilities or mass-weighted abundances.

The original assembly function is replayed with the first 15 and first 16 prime labels in ascending and descending order. The labels {5,47} used to form a neutron candidate are historically fitted inputs. Their outputs are conditional component statistics, not newly derived neutron identities. The audit executes the original address, admission, assembly, charge and color function bodies extracted by Python AST. It bypasses unrelated import-time macro modules; it is a targeted rerun, not the full original Stage-3 pipeline.

Branch	Conditional neutron fraction	Mean component count
K15_small	0.129471986937	15.9763714572
K15_large	0.127159411697	11.4294924701
K16_small	0.124982781217	15.2321441711
K16_large	0.111240636923	10.5785808287

The K15 ascending/descending number-share distributions differ by total-variation distance 0.20472100116369085. This preserves a real order dependence in the declared assembly. A unique particle decoder does not follow from those rules. All four assemblies preserve the exact integer address reconstruction and normalized component shares. The inherited neutral charge motifs remain exactly neutral using rational arithmetic.

The charge table supplies 16 channels, including up-type charges +2/3, down-type -1/3 and electron -1. The tested motifs are uud+electron, udd, an electron/conjugate pair, and an individually neutral channel. Charges are summed as exact rational numbers. Charge assignments and particle names are supplied inputs in this test.

2. What the original color function supplies

The original color_check() has no parameters. It creates a 3 x 3 x 3 complex tensor, assigns the six signed permutations explicitly, constructs six off-diagonal and two diagonal color matrices, and tests their total action. It never reads N, alpha, h, admission weights, assembly counts or neutron labels. Consequently the original replay establishes compatibility with its supplied color contract. It does not establish that arithmetic selected three colors or the SU(3) algebra.

```
|S> = (|rgb> + |gbr> + |brg>
- |rbg> - |grb> - |bgr>) / sqrt(6)
```

For standard normalized fundamental generators T_a with Tr(T_a T_b)=delta_ab/2, let Q_a=T_a tensor I tensor I + I tensor T_a tensor I + I tensor I tensor T_a. The original function uses matrices twice this normalization; annihilation and the common kernel are unchanged.

The eight-matrix fundamental SU(3) basis and the epsilon contraction defining the baryon color singlet are established representation-theory background; see David Tong, The Standard Model, chapter 3, The Strong Force. They are not claimed as new mathematics. The new work here is the source-dependency inspection, the pinned targeted replay and explicit application of the rank-one restriction to this WRRA interface. Collective rotation invariance also follows directly from the determinant-one identity U tensor U tensor U acting on the epsilon tensor.

The intended readers are implementers of the WRRA particle/component interface. The audit changes a concrete development decision: passing the current stand-alone singlet test cannot justify deleting address-dependent structure or calling the color group an arithmetic output. An implementer must instead specify and test a coupling, or retain the color sector as an adopted contract. This is model-specific implementation knowledge; its value does not depend on claiming new SU(3) representation theory.

Throughout this note, independent construction means a separately implemented matrix calculation within the same audit, rather than independent authorship or external experimental replication. No outside laboratory validation is asserted.

dependency_audit.py additionally verifies the frozen source hash, recursively inspects external reads in the compiled function and nested comprehensions, and executes the extracted function with only NumPy, itertools, math and five required builtins. The complete external-name set is {complex, float, itertools, math, max, np, range, sum}; there is no captured closure state or arithmetic global read. All four dependency checks pass. This assumes ordinary unmodified installed libraries; it is not a sandbox against malicious module replacement. These additional provenance checks are separate from the original 36-check audit.

Input dependencies

Input branch	Actual consumer / conclusion
Finite address weights and admission	Prime assembly and component statistics
Inherited charge table and motifs	Rational neutrality checks
Three colors, signed permutations, eight matrices	color_check() constructs and checks them internally; no arithmetic input is read
Exact singlet projector and test joint state	Kernel/factorization audit of the declared interface

Sources and mathematical background

Original Stage-3 code and handoff: github.com/Wonsik-Choi-janefather/wrra-m-0.1/tree/9d9daf21154e46e55b9fd0fd461c7c070156ab94/mcc_3_0/stage_3_particle_component_replay. Parent calculation: calculations/wrra_m_0_9. Frozen exploratory branch cited by Stage 3: DOI 10.5281/zenodo.23128482. These establish computational provenance, not independent experimental validation.

David Tong, The Standard Model, chapter 3: The Strong Force, University of Cambridge lecture notes, equations (3.4)-(3.6) and (3.95). Direct chapter: https://davidtong.org/pdfs/teaching/standard-model/standardmodel3.pdf (accessed 20 October 2026). This supplies standard SU(3) and baryon-singlet background; the conditional proof in section 3 is self-contained.

3. Conditional restriction on address-color coupling

Proposition. Let H_A be any finite address Hilbert space, H_C=(C^3) tensor (C^3) tensor (C^3), and let the three factors carry the same fundamental SU(3) action. Suppose a joint density operator Omega is supported on the exact zero-total-color subspace: Omega=(I_A tensor P) Omega (I_A tensor P), where P projects onto the common kernel of all eight Q_a. Then P=|S><S| and there is a unique address density operator sigma_A such that

```
Omega = sigma_A tensor |S><S|.
```

Proof of the common kernel. In the color basis, the two diagonal traceless generators require equal occupation of r, g and b in each surviving three-factor basis tensor. Only the six permutations of rgb can remain. For each pair of colors i,j, the off-diagonal raising operator E_ij acts on the two permutations that differ by exchanging those colors in their tensor positions, producing the same repeated-color output. Its vanishing requires their coefficients to have opposite signs. These transposition relations connect all six permutations, so all coefficients are fixed by one amplitude and permutation parity. Normalization gives |S>. Direct substitution verifies every Q_a|S>=0. Thus the kernel is one-dimensional.

Proof of factorization. Expand Omega in address matrix units: Omega=sum_nm |n><m| tensor C_nm. Support on the rank-one color projector implies C_nm=P C_nm P=c_nm P. Therefore Omega=sigma_A tensor P, with sigma_A=[c_nm]. Positivity and unit trace of Omega give positivity and unit trace of sigma_A; partial trace over color proves uniqueness. This is a finite linear-algebra consequence of the declared representation and constraint, not a new general theorem about QCD.

4. Implication for the WRRA interface

For any address observable B and color observable C, the expectation becomes Tr(sigma_A B) <S|C|S>. An observable acting only on this color sector cannot distinguish different address distributions. More strongly, a controlled transformation U=sum_n |n><n| tensor (U_n tensor U_n tensor U_n), with every U_n in SU(3), acts as the identity on the exact singlet-supported sector. Such collective color rotations cannot transfer address distinctions into this sector.

Concrete implementation decision. Consider two address preparations sigma_A=|a><a| and sigma_A=|b><b|, a different from b, each tensored with the exact singlet projector. A proposed decoder that deletes the address register and recovers the two different component labels from a color-only observable cannot work: both reduced color states are exactly |S><S|, so every such query has the same answer. Address-controlled collective SU(3) rotations do not repair the loss because each leaves the singlet invariant. Therefore an implementation must retain the address/component register when those labels are requested; passing the standalone color test is insufficient grounds to discard it. This failed-decoder example is prospective implementation guidance under the existing contract, not a reported failure in deployed software or a new particle mechanism.

This result narrows the implementation task. To make an arithmetic update change a color-sensitive output, an additional executable coupling or larger state space is needed. Possible choices include multiple singlet copies in a larger representation, retained spatial or flavor factors, or explicitly tracked nonsinglet intermediate states with a declared admissibility rule. This note selects none of these possibilities as physical and claims no completion of spin, flavor, gauge fields, binding or confinement.

The restriction concerns exact singlet support. It does not follow merely from zero mean color charge, which is weaker. It also does not apply unchanged to a color subsystem entangled with gluons or other omitted factors. These distinctions are part of the stated scope.

5. Independent numerical and exact checks

Diagnostic	Result
Original singlet norm	1.0000000000000002
Maximum total-generator action	0 in the recorded floating-point run
Generator Gram-matrix error	1.110e-16
Commutator-closure residual	1.241e-16
Common kernel dimension	1, numerical and exact symbolic
Collective SU(3) rotation residual (32 trials)	5.847e-16
Joint-state projection/factorization error	1.507e-16
Product rgb state: maximum total-charge norm	0.707106781
One-site phase error: maximum total-charge norm	0.816496581

The negative controls deliberately violate the singlet contract and are detected. They show that the tests can reject specific non-singlet states, rather than returning zero for every input. Exact symbolic nullspace calculation uses nonzero rescalings of the eight generators to eliminate square roots; this preserves the common kernel and verifies the signed integer singlet vector without floating-point tolerance. The factorization diagnostic projects a seeded arbitrary 6-address by 27-color positive density operator onto the singlet sector, normalizes it, and compares it with the tensor product of its address marginal and the fixed singlet projector.

The 36 checks are grouped by purpose: five finite-source/assembly checks; fifteen comparisons with frozen component outputs; one exact charge-motif check; nine generator, kernel, rotation and reduced-state checks; two symmetry-breaking negative controls; three exact-kernel/factorization checks; and one source-hash check. Several test related identities or replay the same source, so the count is a test-suite status rather than 36 independent pieces of scientific evidence.

6. Reproduction and claim boundary

Run from the package directory, preserving source/ alongside audit.py. Recorded environment: Python 3.12.14, NumPy 2.3.5, SciPy 1.17.0 and SymPy 1.14.0. Reproduce with:

```
python3 -m venv .venv
.venv/bin/python -m pip install numpy==2.3.5 scipy==1.17.0 sympy==1.14.0
.venv/bin/python audit.py
.venv/bin/python dependency_audit.py
```

audit_results.json records the runtime versions, thresholds, inputs, declared checks and full output values. SOURCE_SHA256.json is verified before execution and is not overwritten; a changed or missing source makes execution fail. Rerunning writes audit_results.json, and differing numerical-library versions or runtime metadata may change its file hash. Preserve the distributed reference output when checking release hashes and use the recorded tolerances for numerical comparisons. The scientific calculation is separate from PDF rendering.

Exact audit release: WRRA-COLOR-AUDIT-20261020-v0.6. The final manuscript and complete frozen package are archived at https://doi.org/10.5281/zenodo.23496836. This DOI identifies this v0.6 PDF and its companion package. RELEASE.json records SHA-256 identities of the manuscript, scientific scripts, dependency audit, frozen-source manifest and reference outputs. The unchanged scientific calculation also has the pinned GitHub code provenance cited above; the DOI is the primary access route for the exact present release. The supplementary source-dependency check and prospective interface example keep the same particle/color research scope.

Conclusion. The WRRA component replay is reproducible under its frozen inputs and its supplied color-singlet contract. The exact rank-one color sector imposes a concrete interface restriction: it carries no independent address-color correlations. A future arithmetic color mechanism must explicitly overcome that restriction through a stated extension. This is a targeted audit and model-development result; publication suitability is left to independent review.